

Graph-Transformer-based Surrogate Model for Accelerated Converter Circuit Topology Design

Shaoze Fan¹, Haoshu Lu¹, Shun Zhang², Ningyuan Cao³, Xin Zhang², and Jing Li¹

¹New Jersey Institute of Technology ²MIT-IBM Watson AI Lab ³University of Notre Dame

Abstract

Unlike circuit parameter and sizing optimizations, the automated design of analog circuit topologies poses significant challenges for learning-based approaches. One challenge arises from the combinatorial growth of the topology space with circuit size, which limits the topology optimization efficiency. Moreover, traditional circuit evaluation methods are time-consuming, while the presence of data discontinuity in the topology space makes the accurate prediction of circuit performance exceptionally difficult for unseen topologies. To tackle these challenges, we design a novel Graph-Transformer-based Network (GTN) as the surrogate model for circuit evaluation, offering a substantial acceleration in the speed of circuit topology optimization without sacrificing performance. Our GTN model architecture is designed to embed voltage changes in circuit loops and current flows in connected devices, enabling accurate performance predictions for circuits with unseen topologies. Taking the power converter circuit design as an experimental task, our GTN model significantly outperforms an analytical approach and baseline methods directly utilizing graph neural networks. Furthermore, GTN achieves less than 5% relative error and 196× speed-up compared with high-fidelity simulation. Notably, our GTN surrogate model empowers an automatic circuit design framework to discover circuits of comparable quality to those identified through high-fidelity simulation while reducing the time required by up to 98.2%.

1 INTRODUCTION

The rapid advancement of artificial intelligence has led to a new era of innovation in electronic design automation (EDA), particularly in the domain of analog circuit design. The analog circuit design workflow comprises four phases: circuit topology design, physical layout design, parasitic extraction, and validation. While learning-based EDA tools have achieved notable success in the lower-level design phases [6, 12, 18], the challenge of automating circuit topology design persists, predominantly relying on the expertise and experiences of human designers. Moreover, the field of analog circuit design offers abundant opportunities for novel topologies, as demonstrated by the continuous evolution of power converter topologies year after year [8]. Hence, the development of learning-based topology design tools holds immense promise and relevance in this dynamically expanding domain.

*Corresponding authors: jingli@njit.edu, xzhang@us.ibm.com.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0601-1/24/06.

<https://doi.org/10.1145/3649329.3656258>

However, the automatic design of circuit topologies faces formidable challenges. The topology design space grows faster than an exponential increase with the circuit complexity and experiences a combinatorial explosion. For example, Figure 1 shows that the number of circuit topologies rapidly escalates to 191.8K for 5-component circuits, when each component can be chosen from 4 types and have 2 ports to be connected with other component ports or 3 terminal ports. Consequently, while a single simulation for a 5-component circuit may require just a few seconds, evaluating all these diverse topologies, each with multiple device parameters, demands an astonishing 111 days. Recent works have introduced machine learning algorithms tailored for circuit topology automation, encompassing tree-based search [5, 23] and genetic algorithms [13], which intelligently explore the circuit topology space. Nevertheless, the effectiveness of these algorithms hinges on the time cost and accuracy of circuit evaluation. Thus, they still suffer intolerably computational costs when employing high-fidelity simulations to assess the searched circuit designs.

Leveraging learning-based methods as surrogate models to predict circuit performance holds great promise for accelerating circuit evaluation, but it brings about distinctive challenges in the context of circuit topology design. Unlike surrogate models aimed at optimizing device parameters of a fixed topology that remains the same for both training and testing [2], those for topology design must possess the capability to estimate the performance of circuits with topologies previously unobserved in training data, as training a separate model for each topology is impractical for design automation. This necessitates that the model comprehends the inherent topological structures of circuits. Moreover, learning-based surrogate models for topology design suffer from severe data discontinuity. Unlike problems in the general AI field, where similarities in topologies often yield similar predictions, circuit design exhibits the phenomenon wherein even a minor change in topology can dramatically alter circuit characteristics and performance. Additionally, the data imbalance in topology space, due to the fact that good circuit candidates are inherently sparse, aggravates the difficulties in learning accurate prediction. As a result, applying existing models developed in the broader AI field, such as graph neural networks, proves less effective. Instead, learning-based surrogate models for topology design must be engineered to capture the fundamental physics underlying diverse topologies, enabling the transfer of knowledge gleaned from a limited set of topologies to those that are previously unseen. In pursuit of effective circuit topology design automation, we introduce a pioneering Graph-Transformer-based Network (GTN) as our surrogate model, aimed at precise circuit performance prediction and expedited circuit evaluation and topology exploration. Specifically, we make the following contributions.

- We redefine a circuit topology as a topological graph, where circuit devices and their connection nodes are both represented as

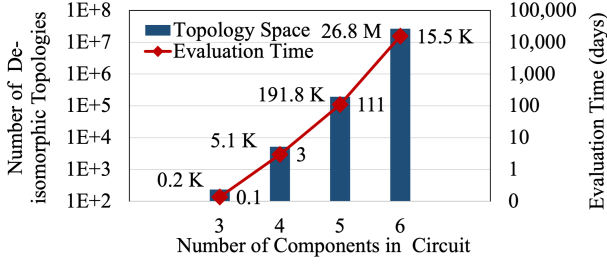


Figure 1: For circuits with 4 different component types, the topology space and circuit evaluation time using high-fidelity simulation increase combinatorially with circuit complexity.

graph vertices. This representation, though resulting in larger graphs compared to those with devices alone as vertices, enables the exact preservation of topological information, including the connectivity of different device ports. This proves pivotal for detecting cases, such as shortcuts arising from certain device port connections, which are crucial in identifying undesirable topologies generated by topology search algorithms. Furthermore, it can truthfully convey current flows among devices linked to the same connection node. To identify voltage changes within the circuit, we employ the minimum loop algorithm [10] to identify minimally overlapping loops where voltage changes can be effectively characterized with the smallest number of devices.

- We construct a novel GTN neural architecture that harnesses the topological knowledge to embed voltage changes in circuit loops and current flows between connected devices. This is achieved by employing multi-head attention mechanisms, both for aggregating embeddings of neighboring devices linked to the same connection node cultivating the current flow information, and for aggregating embeddings of devices within the same circuit loop cultivating the voltage change information. Although circuit loops can capture knowledge essential for accurate circuit performance prediction, their greater distance compared to neighbors makes training with a cold start difficult. Thus, we introduce a specialized learning procedure that starts with training only the aggregation of neighboring vertices via a special masking technique and progressively adds the aggregation of loop devices once the remaining networks have warmed up well.
- We focus on the power converter circuit topology design as an experimental task in this work. Evaluation results show that our GTN model reduces the relative squared error of an analytical approach and baseline methods directly utilizing various graph neural networks by more than 87.7% and 16.1%, respectively. Furthermore, GTN can predict circuits' power efficiency and output voltage with less than 5% relative error and 196× speed-up compared with high-fidelity simulation.
- We integrated our GTN surrogate model into a tree-based search for a circuit topology design automation framework. GTN empowers this framework to discover circuits of comparable quality to those identified through high-fidelity simulation while reducing the time required by up to 98.2%.

2 RELATED WORKS

AI-based Circuit Design Automation. Applying learning-based models for circuit design tasks, such as physical implementation [6,

12] and parameter optimization [18] of integrated circuits, have attracted increasing attention. However, most works focus on circuit optimization using a given circuit topology. For topology exploration, search algorithms have been investigated, including evolutionary, genetic [13], and tree-based search algorithms [5, 23].

Learning-based Circuit Evaluation. Conventional SPICE simulation software takes an extremely long time to evaluate one topology with fixed parameters. Analytical methods, like State-Space Averaging [14] for converter circuits, can reduce the time cost of evaluating one topology with different parameters, but they can only be applied to a small fraction of topologies that satisfy certain theoretical assumptions. Thus, recent works developed deep learning models to speed up circuit performance and characteristic estimation tasks, such as estimating dynamic voltage drop [20, 24], post-layout interconnect parasitics [16], and net lengths [19] of integrated circuits. Various Graph Neural Networks (GNN) were designed for estimating electromagnetic properties [21], power [22], transient electromigration stress [9], Circuit De-obfuscation Runtime [4], as well as guiding analog IC placement [11], layout parasitics and device parameter prediction [15], symmetry constraint extraction [3], reverse engineering [1] and arithmetic block identification [7].

However, most prior works focused on tasks that involve training and testing using a single topology or similar topologies. They tend to perform suboptimally when confronted with the challenges posed by a diverse topology design space with data discontinuity and imbalance. In contrast, this work aims for accurate performance prediction of diverse and unseen topologies explored by topology design automation frameworks.

3 PROBLEM STATEMENT

In this work, we aim to design a fast-running surrogate model f that can learn from a small amount of ground truth circuit evaluation results to accurately predict the key performance of a circuit with unobserved topology generated by an automatic circuit topology design framework. In this way, the surrogate model that can help the design automation framework generate circuits with the same quality as the framework using high-fidelity simulation while reducing the time cost as much as possible. In this context, a circuit consists of a *topology* s and a *device parameter* d . The circuit topology contains a set of device components with ports and edges connecting the ports. Each component has a device type (from the set of available devices in the design task) with device parameters (e.g., inductance, capacitance, transistor dimensions, and switching control parameters) and ports (i.e., left and right ports).

The accuracy of the surrogate model is measured by the Relative Squared Error $RSE = \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$, where $\hat{y}_i$ is the performance predicted by the model, y_i is the ground-truth performance given by the high-fidelity simulation F_{sim} , and $\bar{y}$ is the sample mean of the ground-truth performance. Intuitively, the RSE gives a normalized mean squared error. Thus, our goal is to design a surrogate model f that achieves the minimum RSE:

$$f^* = \arg \min_f \frac{\sum_i (F_{sim}(s_i, d_i) - f(s_i, d_i))^2}{\sum_i (F_{sim}(s_i, d_i) - \bar{F}_{sim})^2}$$

In this paper, we use the two-phase power converter circuit design as the exemplar and experimental design task. Specifically, we

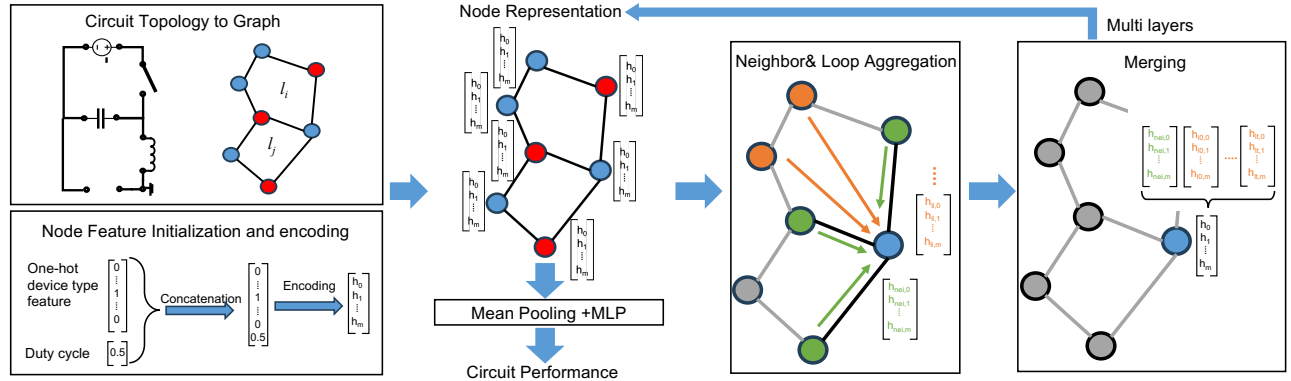


Figure 2: GTN Surrogate Model Structure.

consider 5-components circuits with devices types of capacitors (C), inductors (L), phase-I switches (S_a), and phase-II switches (S_b). Each component has two ports without direction. In addition to components, three external terminal ports are essential in each circuit: the input voltage (V_{in}), output voltage (V_{out}), and ground (Gnd) ports. For device parameter d , we consider the duty cycle of switches that affects the output voltage. We consider two key performance metrics of power converters, the output voltage $V_{out}(s, d)$ and power conversion efficiency ($\eta_{s,d}$). When automatically design power converter circuits, a high-quality converter is the one that has high efficiency and V_{out} close to a target output voltage V_0 .

4 GTN SURROGATE MODEL

In this section, we present our topological graph representation of circuits and the insights from Kirchhoff’s Current and Voltage Laws that are exploited in the novel model architecture of our Graph-Transformer-based Network (GTN) surrogate model.

4.1 Topological Graph Representation

In contrast to existing learning-based surrogate models that represent only the devices as vertices in a circuit’s topological graph, our approach redefines the topological graph to include both devices and their connection nodes as graph vertices. While the graph representation with devices alone yields smaller graphs and expresses device connectivity, it lacks the vital information regarding which ports of devices are connected. This omission results in identical graph representations for circuits with different performances, such as high-quality circuits and those with shortcuts. In contrast, our topological graph representation (akin to the netlist representation) preserves precise topological information, proving essential in identifying undesirable topologies generated by topology search algorithms. Additionally, it accurately conveys current flows within the circuit, a key characteristic leveraged in our GTN architecture.

As described in Section 3, each device component is associated with a specific device type, while a connection node can either be one of the terminal ports or an internal connection. To effectively capture these distinct vertex types in the topological graph, we employ a one-hot representation as the vertex feature, where each bit corresponds to a particular device or connection type. Additionally, we concatenate the device parameters (normalized to be at the same scale of 1) into the vertex feature, where the parameter is set to 0 if the vertex is not the corresponding device type.

Furthermore, we supplement our topological graph with explicitly identified circuit loops, which are utilized in our GTN model to capture the conservation of charge and energy in circuit loops. While all circuit loops hold voltage information, they encompass largely overlapping voltage data. Moreover, using all circuit loops makes the model more complex and harder to train. To address this challenge, we leverage the minimum loop algorithm [10] implemented in the Networkx package to identify minimally overlapping loops (e.g., l_i and l_j in Figure 2). This strategic approach allows us to accurately characterize voltage information with a minimal number of devices, enhancing the GTN model’s effectiveness.

4.2 Physics Insights Behind GTN Architecture

Real-world circuits often exhibit inherent complexity, rendering direct analysis challenging. It is also challenging for learning-based surrogate models to grasp their underlying physics if blindly employing standard deep learning models. To facilitate the model’s comprehension of critical physics information embedded within circuit topologies, we draw inspiration from Kirchhoff’s two fundamental laws of electrical circuits, which encapsulate the unique interplay of current, voltage, and resistance in electrical circuits. Specifically, Kirchhoff’s Current Law dictates that the current entering a connection node or device within a circuit equals the current departing from the same node or device. Kirchhoff’s Voltage Law states that, for any closed loops within an electrical circuit, the sum of the electrical voltage across the loop must equate to zero.

Our approach to incorporating these physics insights involves two key steps. Firstly, we explicitly encode the connected nodes (in addition to devices) as graph vertices and identify smallest loops within our topological graph representation. Subsequently, we tailor the architecture of our GTN model to aggregate the information within the small sub-circuits of interconnected vertices and smallest loops, which captures the nuances of charge and energy conservation. This synergistic integration bolsters GTN’s capability in unraveling the intricate physics underpinning complex circuit topologies, enhancing its effectiveness in modeling circuits.

Note that the state-space averaging method [14] for analyzing power converter circuits is also built upon Kirchhoff’s two laws. However, it solves the state-space equations based on the assumption that the current or voltage on each device is stable. The equation matrix, which is a canonical form for writing the differential equations of a circuit, presents challenges for certain circuit (e.g.,

circuits containing inductors connected in series). Our empirical evaluation of state-space averaging for 5-component circuits shows that 46% circuits cannot derive the canonical matrix. Additionally, among circuits that can derive the canonical matrix, an additional 7% were unable to derive the inverse matrix, a prerequisite for the state space averaging method. All these circuits have diverse performance and cannot be set to a constant performance value.

4.3 GTN Model Architecture

Our innovative GTN neural architecture, depicted in Figure 2, leverages topological knowledge and physics laws of voltage changes in circuit loops and current flows between connected devices. Specifically, it first embeds the vertex features detailed in Section 4.1 into vertex embeddings and then employs multiple layers that share a common architecture but do not share weights. Each layer consists of 3 steps: neighboring vertex aggregation, loop device aggregation, and merging. Intuitively, the neighboring vertex aggregation step can assimilate embedded information concerning current flows arriving and departing a connection node or device. Meanwhile, the loop device aggregation step aggregates embedded information pertaining to voltage changes across circuit loops. Harmoniously combining these embedded information facilitates learning the topological and physical characteristics of circuit topologies.

The neighbor and loop aggregations generate the aggregated embedding using the multi-head attention (MHA) mechanism in Transformer [17], and the merging step combines these two embeddings into the embedding of each topological graph vertex.

Concretely, we use *MHA* to denote the attention calculation:

$$MHA(e_i, e_j) = softmax(\frac{e_i \mathbf{W}_Q \cdot e_j \mathbf{W}_K}{\sqrt{D_k}}) e_j \mathbf{W}_V \quad (1)$$

where e_i and e_j are two embeddings, $\mathbf{W}_Q \in \mathbb{R}^{D_{in} \times D_k}$, $\mathbf{W}_K \in \mathbb{R}^{D_{in} \times D_k}$ and $\mathbf{W}_V \in \mathbb{R}^{D_{in} \times D_{out}}$ are the query, key and value matrices, respectively. And D_{in} , D_{out} and D_k are the matrix dimensions.

The neighbor aggregation step generates the neighbor embedding h_{nei} for each vertex n by aggregating the embeddings of neighboring vertices $Nei(n)$ of vertex n using multi-head attention:

$$h_{nei}^n = \sum_{m \in Nei(n)} MHA_{nei}(h^n, h^m) \quad (2)$$

The loop aggregation step generate one loop embedding $h_{l_i}^n$ for each device or terminal port vertex n in each loop l_i using MHA:

$$h_{l_i}^n = \sum_{m \in Loop(l_i)} MHA_l(h^n, h^m) \quad (3)$$

in which $Loop(l_i)$ denotes the set of device and terminal port vertices that are in the loop l_i . Note that the internal connections nodes that do not have voltages do not participate the loop aggregation.

In the merging step, after getting the neighbor embedding h_{nei}^n and all the loop embeddings $h_{l_0}^n, h_{l_1}^n, \dots, h_{l_t}^n$, where t is the total number of loops that vertex n is involved in, we combine all these embeddings using a weighted summation function:

$$h^n = \sum_{h \in (h_{nei}, h_{l_0}^n, h_{l_1}^n, \dots, h_{l_t}^n)} softmax(\frac{h \mathbf{W}_Q \cdot h \mathbf{W}_K}{\sqrt{D_k}}) h \quad (4)$$

The 3 steps generate the embedding of each vertex in one layer, and we iterate these processes for multiple layers. Lastly, as a graph-level prediction task, we conduct the mean pooling to generate

the representation of the entire circuit and use a multi-layer fully connected network to generate the final performance prediction.

4.4 Sub-network Warm Up

Although circuit loops can capture knowledge essential for accurate prediction, their larger distance leads to over-smoothing and makes training with a cold start difficult. In comparison, the neighbor embedding is more local and easier to learn. Thus, we introduce a specialized learning procedure that starts with training only the aggregation of neighboring vertices via a special masking technique applied to the merging step (omitted due to space limit). Only after the remaining networks have warmed up well, do we remove the masking and add the aggregation of loop devices.

5 EVALUATION

This section evaluates our Graph-Transformer-based Network (GTN) surrogate model for circuit design. We first show the prediction accuracy and speed of GTN. Then, we compare GTN with different baseline surrogate models. To comprehensively assess the advantage of our GTN model in enhancing circuit design, we integrate GTN into a circuit generation framework and compare it with the framework using the software simulator. The experiments are conducted on two servers: one has an Intel Xeon 6238R CPU with two NVIDIA RTX A4000 GPUs. The other has an Intel(R) Xeon(R) Silver 4216 CPU with two NVIDIA RTX A5000 GPUs.

5.1 Experiment Setup

Dataset. We conduct the evaluation on the power converter design task with 5 components. The total number of vertices of the corresponding topological graphs is often larger than 13 with internal connection nodes and 3 terminal ports. The capacitance is set as C ($10\mu F$) and the inductance as L ($100\mu H$). The frequency is $1MHz$ and the input voltage is $100V$. For external devices, we consider an input resistor of 0.1Ω for V_{in} , and an output resistor of 100Ω and an output capacitor of $10\mu F$ for V_{out} . The duty cycle ranges from 0.1 to 0.9, with a step size of 0.2. We exhaustively generate 959k converter circuits (with 191.8k topologies) that cover almost all those with valid outputs. Note that the circuits with the same topology but different duty cycles are considered distinct. We evaluated their efficiency and output voltage using the NGSPICE-38 simulator as ground truth. Regarding the model training, we randomly sample from the entire dataset at 6 various ratios to construct separate training datasets. The sampling ratios spanned from 0.33% to 1.98%, with an increment of 0.33%. We randomly sample 1% of the dataset as the test dataset and 0.15% of the dataset for validation purposes.

GTN Implementation. For topological graph representation, we convert the circuit topology into a graph and encode vertices into embeddings with dimension of 48, which is the size of D_{in} and D_{out} . Then, we identify loops using Networkx. We store the graph and loops in the format of adjacency matrix, which can simplify the mask operations in the training. As for the model, we construct GTN by comprising a total of 4 layers with ReLU activation. For the multi-head attention mechanism within each layer, we configure the number of heads as 4 and D_k as 6. The MLP network for prediction has two hidden layers with sizes of 64 and 16, respectively. In this way, the total number of parameters in the GTN is around 157k.

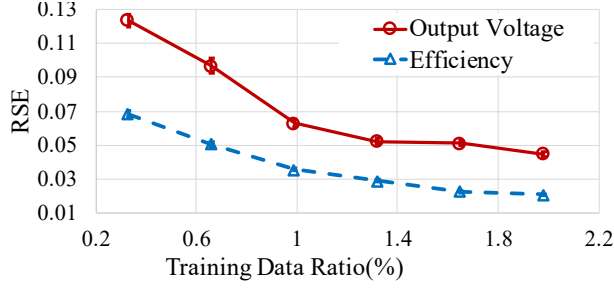


Figure 3: Relative Squared Error with increasing training data ratios. The more training data used, the higher the accuracy.

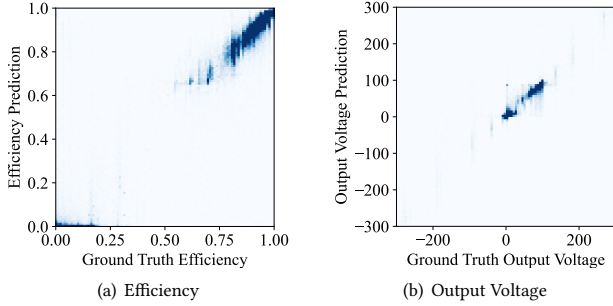
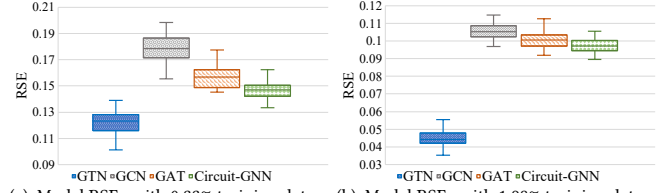


Figure 4: Prediction distribution of the GTN for converter circuits. The more data points cluster around the diagonal line, the closer the prediction to the ground truth, and the better the surrogate model performs.

We train each model for 700 epochs in total and aggregate the loop devices after 450 epochs of sub-network warm up.

Baseline Models. We compare our GTN model with three baseline models: Graph Convolutional Network (GCN), Graph Attention Network (GAT) [19], and Circuit-GNN [21]. GCN updates a node’s feature by combining its feature with features from neighboring nodes. Similarly, GAT aims to learn node representations in a graph but dynamically calculates neighboring node weights using an attention mechanism. Different from GCN and GAT, Circuit-GNN [21] is designed to learn to simulate the electromagnetic properties of distributed circuits with a special GNN model that captures information from both nodes and edges across the entire graph.

Experiments of Circuit Generation. We integrated our GTN surrogate model into the framework in [5] that generates custom power converters from design specifications using Monte Carlo Tree Search (MCTS). To demonstrate the advantage of the GTN model, we conduct a comparative analysis with the same framework but incorporate the high-fidelity NGSpice simulator for evaluation. The simulator plays the same role as the surrogate model in the framework. Most experiment settings for circuit generation are the same as the ones in [5]. The target output voltage is set as 50V, equivalent to the 0.5 target conversion ratio in [5]. Major differences include (1) values of the duty cycle are from 0.1 to 0.9, with a step size of 0.2; (2) the circuit evaluation is queried for each topology with a specific duty cycle but not duty cycle sweeping; (3) we query the top 1% of the generated circuits for high fidelity evaluation.



(a) Model RSEs with 0.33% training data. (b) Model RSEs with 1.98% training data. **Figure 5: Comparison of our GTN with baseline models (GCN, GAT, and Circuit-GNN) for output voltage prediction of power converter circuits – all models benefit from more training data, while GTN significantly outperforms baselines.**

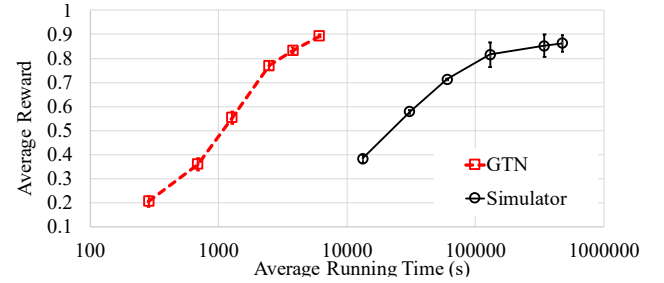


Figure 6: Performance comparison of GTN and simulator in enhancing circuit design. x-axis: the log-running time. y-axis: reward of circuits. *GTN*: generation framework with GTN. *Simulator*: the framework with NGSpice simulator.

5.2 Experiment Results

Performance of GTN Surrogate Model. Figure 3 shows the performance of our model using different training ratios. Trained with 40 seeds, each model calculates the average RSE on the testing set. As the training set expands, the RSEs for efficiency and output voltage monotonically decrease. When we use 1.98% of the dataset for training, the RSE of the efficiency is 2.08% and the output voltage is 4.48%. Figure 4 visualize the distribution of efficiency and output voltage generated by the GTN versus the corresponding ground-truth value on the entire dataset. 4(a) and 4(b) shows that most data points are aligned with the diagonal line. This confirms that our GTN can precisely evaluate the entire dataset and is accurate enough for circuit design.

As for the computation time, we measured the average inference time, that is, the time needed to make a prediction given a circuit. The average inference time is 0.56 sec. over 16k data points. The NGSpice simulation time is 111.7 sec. The GTN achieves over 196× speed-up, highlighting its computational efficiency advantage.

In conclusion, our GTN is a fast-running surrogate model that can learn from a small amount of ground truth circuit evaluation and accurately predict the key performance of unseen circuits.

Comparison with Baselines. We compare the GTN with the baseline models for predicting circuit output voltage. Fig.5(a) shows RSE of 0.33% training data and Fig.5(b) shows the ones with 1.98% training data. We observe that the GTN achieves 16.1% and 53.9% performance improvement respectively compared to Circuit-GNN, and Circuit-GNN performs better than GAT and GAT performs better than GCN. The superiority of GTN and the performance trend remain consistent across different experimental configurations and

datasets. We also compute the RSE when using the state-space averaging method. When setting the voltage prediction of the circuits that cannot be analyzed by state-space averaging as 0 (i.e., disconnect) or 1 (i.e., shortcut), the RSE of state-space averaging is at least 1.0. In comparison, the RSE of our GTN model is only 0.123, achieving a relative improvement of 87.7%.

The results demonstrate that our GTN model, utilizing the multi-head attention mechanism to aggregate both neighbor and loop information, outperforms baseline models focusing solely on neighbor aggregation. Our model can effectively represent nodes within the graph and capture crucial current and voltage knowledge essential for accurate circuit performance prediction.

Accelerating Circuit Generation using GTN. Figure 6 shows the performance of the power converter generation frameworks using different circuit evaluation methods. The reward R for each converter circuit is defined as:

$$R(s, d, V_0) = \eta_{s,d} \cdot 1.1^{-\left(\frac{15(V_{\text{out}}(s,d) - V_0)}{|V_0|}\right)^2}. \quad (5)$$

Recall that $V_{\text{out}}(s, d)$ is the output voltage and $\eta_{s,d}$ denotes the power conversion efficiency. A high-quality converter has high efficiency and V_{out} close to a target output voltage V_0 which is 50V in our experiment. Thus higher R means better quality of a power converter circuit. We run each set 200 times and output the average reward of the generated circuits. Compared with the simulator, the proposed GTN surrogate model significantly reduces the overall design time by up to 98.2% performance without degradation of the quality of the generated power converters. This improvement shows that, during the circuit topology generation process, GTN can swiftly and accurately predict the efficiency and output voltage of the generated circuit. Consequently, our GTN model guarantees that the framework keeps sufficiently exploring the space with better circuit designs. In conclusion, the results demonstrate the effectiveness of the proposed GTN surrogate model that can accurately evaluate the performance of the circuits with a much lower time cost compared to the high-fidelity simulator.

6 CONCLUSION

This work introduces a novel Graph-Transformer-based Network (GTN) surrogate model to accelerate circuit evaluation and topology exploration without sacrificing performance. GTN cultivates voltage information in circuit loops and current information in connected devices, enabling accurate predictions for unseen topologies. For power converter circuit design, GTN outperforms analytical and baseline methods, reduces the design time by up to 98.2%, and offers a powerful tool for automatic topology design. In the future, we plan to enhance GTN by incorporating additional factors, such as layout parasitics, apply GTN to larger-scale power converters, and explore its utility in designing other types of circuit topologies.

ACKNOWLEDGMENTS

This material is based upon work supported in part by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research under Award Number DE-SC0024424 and the Advanced Research Projects Agency-Energy (ARPA-E) under Award Number DE-AR0001210, and by the U.S. National Science Foundation under Grant Number CNS-1948457. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States Government or any agency thereof.

References

- [1] L. Alrahis, A. Sengupta, J. Knechtel, S. Patnaik, H. Saleh, B. Mohammad, M. Al-Qutayri, and O. Sinanoglu. 2021. GNN-RE: Graph neural networks for reverse engineering of gate-level netlists. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 8 (2021), 2435–2448.
- [2] V. Bui, F. Chang, W. Su, M. Wang, Y. Murphey, F. Da Silva, C. Huang, L. Xue, and R. Glatt. 2022. Deep Neural Network-Based Surrogate Model for Optimal Component Sizing of Power Converters Using Deep Reinforcement Learning. *IEEE Access* 10 (2022), 78702–78712.
- [3] H. Chen, K. Zhu, M. Liu, X. Tang, N. Sun, and D. Pan. 2021. Universal symmetry constraint extraction for analog and mixed-signal circuits with graph neural networks. In *ACM/IEEE Design Automation Conference (DAC)*. 1243–1248.
- [4] Z. Chen, G. Kolhe, S. Rafatirad, C. Lu, S. PD, H. Homayoun, and L. Zhao. 2020. Estimating the circuit de-obfuscation runtime based on graph deep learning. In *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 358–363.
- [5] S. Fan, N. Cao, S. Zhang, J. Li, X. Guo, and X. Zhang. 2021. From Specification to Topology: Automatic Power Converter Design via Reinforcement Learning. In *IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9.
- [6] K. Hakhamaneshi, N. Werblun, P. Abbeel, and V. Stojanović. 2019. BagNet: Berkeley analog generator with layout optimizer boosted with deep neural networks. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 1–8.
- [7] Z. He, Z. Wang, C. Bail, H. Yang, and B. Yu. 2021. Graph learning-based arithmetic block identification. In *IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, IEEE, 1–8.
- [8] A. Jackson, N. Ellis, and R. Pilawa-Podgurski. 2022. A Capacitively-Isolated Dual Extended LC-tank Hybrid Switched-Capacitor Converter. In *IEEE Applied Power Electronics Conference and Exposition (APEC)*. 1279–1283.
- [9] W. Jin, L. Chen, S. Sadiqbatcha, S. Peng, and S. Tan. 2021. EMGraph: Fast Learning-Based Electromigration Analysis for Multi-Segment Interconnect Using Graph Convolution Networks. In *Design Automation Conference (DAC)*. 919–924.
- [10] T. Kavitha, K. Mehlhorn, D. Michail, and K. Paluch. 2008. An algorithm for minimum cycle basis of graphs. *Algorithmica* 52, 3 (2008), 333–349.
- [11] Y. Li, Y. Lin, M. Madhusudan, A. Sharma, W. Xu, S. Sapatnekar, R. Harjani, and J. Hu. 2020. A customized graph neural network model for guiding analog IC placement. In *International Conference On Computer Aided Design (ICCAD)*. 1–9.
- [12] Y. Lu, S. Pentapati, L. Zhu, K. Samadi, and S. Lim. 2020. TP-GNN: A Graph Neural Network Framework for Tier Partitioning in Monolithic 3D ICs. In *ACM/IEEE Design Automation Conference (DAC)*. 1–6.
- [13] T. McConaghy, P. Palmers, M. Steyaert, and G. Gielen. 2011. Trustworthy Genetic Programming-Based Synthesis of Analog Circuit Topologies Using Hierarchical Domain-Specific Building Blocks. *IEEE Transactions on Evolutionary Computation* 15, 4 (2011), 557–570.
- [14] W. M. Polivka, P. R.K. Chetty, and R. D. Middlebrook. 1980. State-Space Average modelling of converters with parasitics and storage-time modulation. In *1980 IEEE Power Electronics Specialists Conference*. 119–143.
- [15] H. Ren, G. Kokai, W. Turner, and T. Ku. 2020. ParaGraph: Layout parasitics and device parameter prediction using graph neural networks. In *ACM/IEEE Design Automation Conference (DAC)*. 1–6.
- [16] B. Shook, P. Bhansali, C. Kashyap, C. Amin, and S. Joshi. 2020. MLParest: Machine Learning based Parasitic Estimation for Custom Circuit Design. In *ACM/IEEE Design Automation Conference (DAC)*. 1–6.
- [17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N Gomez, L. Kaiser, and I. Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [18] H. Wang, K. Wang, J. Yang, L. Shen, N. Sun, H. Lee, and S. Han. 2020. GCN-RL circuit designer: Transferable transistor sizing with graph neural networks and reinforcement learning. In *ACM/IEEE Design Automation Conference (DAC)*. 1–6.
- [19] Z. Xie, R. Liang, X. Xu, J. Hu, Y. Duan, and Y. Chen. 2021. Net2: A Graph Attention Network Method Customized for Pre-Placement Net Length Estimation. In *Asia and South Pacific Design Automation Conference (ASP-DAC)*. 671–677.
- [20] Z. Xie, H. Ren, B. Khailany, Y. Sheng, S. Santosh, J. Hu, and Y. Chen. 2020. PowerNet: Transferable Dynamic IR Drop Estimation via Maximum Convolutional Neural Network. In *Asia and South Pacific Design Automation Conference (ASP-DAC)*. 13–18.
- [21] G. Zhang, H. He, and D. Katabi. 2019. Circuit-GNN: Graph neural networks for distributed circuit design. In *International Conference on Machine Learning*. PMLR, 7364–7373.
- [22] Y. Zhang, H. Ren, and B. Khailany. 2020. GRANNITE: Graph Neural Network Inference for Transferable Power Estimation. In *ACM/IEEE Design Automation Conference (DAC)*. 1–6.
- [23] Z. Zhao and L. Zhang. 2020. An Automated Topology Synthesis Framework for Analog Integrated Circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 12 (2020), 4325–4337.
- [24] H. Zhou, W. Jin, and S. Tan. 2020. GridNet: Fast Data-Driven EM-Induced IR Drop Prediction and Localized Fixing for On-Chip Power Grid Networks. In *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9.

Order-Preserving Encryption for the Confidential Inference in Random Forests: FPGA Design and Implementation

Rupesh Raj Karn
Khalifa University
Sas Al Nakhl, Abu Dhabi, UAE

Kashif Nawaz
Technology Innovation Institute
Masdar City, Abu Dhabi, UAE

Ibrahim (Abe) M. Elfadel
Khalifa University
Sas Al Nakhl, Abu Dhabi, UAE

ABSTRACT

Prior work has addressed the problem of confidential inference in decision trees. Both traditional order-preserving cryptography (OPE) and order-preserving NTRU cryptography have been used to ensure data and model privacy in decision trees. Furthermore, FPGA architectures and implementations have been proposed for implementing such confidential inference algorithms on resource-limited, edge-based platforms such as low-cost FPGA boards. In this paper, we address the challenging problem of scalability of order-preserving confidential inference to random forests, which are ensembles of decision trees that are meant to improve their classification accuracy and reduce their overfitting. The paper develops a methodology and an FPGA implementation strategy for scaling up OPE to random forests. In particular, a framework is used to study the multifaceted tradeoffs that exist between the number of trees in the random forest, the strength of the encryption, the accuracy of the inferences, and the resources of the edge platform. Extensive experiments are conducted using the MNIST dataset and the Intel DE10 Standard FPGA board.

KEYWORDS

Decision Tree, Random Forest, Order-preserving Encryption, Confidential Inference, FPGA, Sequential Circuit, Combinational Circuit

1 INTRODUCTION

The aim of privacy-preserving machine learning [1–3] is to reconcile the divergence between privacy concerns and the advantages of machine learning (ML). This paper illustrates an approach to address ML classification challenges by employing a fusion of random forest (RF) and privacy methodologies. Model privacy ensures that a malicious entity cannot compromise the model. Given that ML models often represent significant intellectual property for many companies, the risk of model theft carries substantial financial risks. On the other hand, data privacy ensures that an adversarial entity cannot reverse-engineer the training/validation data. Recent research [4] has shown that reconstructing training data and reverse engineering models are more feasible than commonly perceived. Privacy-preserving ML techniques encompass perturbation methods such as differential privacy [5], cryptographic approaches such as homomorphic encryption and multiparty computing [6], as well as ML-specific strategies like federated learning [2]. In the context of RFs, previous works [1–3, 6, 7] has used various homomorphic encryption techniques to protect privacy, during both the training and inference phases. In this paper, we use OPE, a cryptographic algorithm that is well suited for RFs [8, 9]. We focus on ensur-

ing the privacy of the inference phase on an edge platform where the confidentiality of the provider's model and user's data is most challenging to achieve.

In [10], it is demonstrated that preserving the confidentiality of decision tree inference is attainable through the use of traditional OPE [8]. In particular, the mathematical prerequisites for this approach are considerably less stringent compared to fully homomorphic cryptography [11]. To address computational complexity, a lightweight homomorphic encryption tailored for decision tree inference is presented in [12]. While [10, 12] is effective for decision trees, the hardware design for ensembles of trees and their FPGA implementation remains unexplored. High-level synthesis (HLS) tools such as HLS4ML and Conifer are available to generate hardware designs for Xilinx's FPGA [13, 14]. However, these tools are based on Xilinx IP and require Vivado HLS for design generation, making them incompatible with Intel's FPGA. Furthermore, these HLS tools lack APIs for incorporating cryptographic modules. Following [8], significant advances have been made to enhance the security of OPE against chosen-plaintext attacks. Evaluating the adaptation of the latest OPE to decision trees and RFs is crucial. Furthermore, the constraints in selecting between successive and parallel tree evaluations have not been thoroughly investigated.

The above open problems motivate our own exploration of the latest OPE methods, their relevance to privacy preservation within the RF context, the hardware design for trees ensembles, and ultimately the implementation on FPGA. We devise the digital circuit design for the RF model without relying on any IP core, ensuring platform independence, which enables operations on both Xilinx and Intel FPGA. Moreover, it is crucial to present the trade-off between RF inference accuracy and FPGA resource utilization to comprehensively assess the hardware design and its overhead. In this study, we strive to address these aspects. Our contributions are the following: (1) we propose a novel OPE algorithm for the confidential inference on RFs, and (2), we evaluate an FPGA implementation of OPE-encrypted RF on an Intel FPGA and analyze the tradeoffs between inference accuracy, security strength, and FPGA resources.

The remainder of this paper is organized as follows. In Section 2, we provide background on OPE and its application to random forests. In Section 3, the hardware circuit design of encrypted random forest and its synthesis on FPGA is described along with the communication protocols that we have set up to obtain inference on the FPGA. The hardware overhead, numerical results, and discussion of encryption strength are given in Section 4. The paper concludes in Section 5.

2 ENCRYPTION OF RANDOM FOREST

2.1 Order-Preserving Encryption

An order-preserving function f mapping one set $\{0, \dots, R_p\}$ to another $\{1, \dots, R_c\}$, where $c > p$, can be uniquely expressed by a selection of p out of c ordered items. R_p and R_c denote the ranges of

plaintexts and ciphertexts, respectively. The encryption function's key consists of two components:

- (1) the probability distribution of the ciphertexts $\{0, \dots, R_c\}$,
- (2) the unique mapping $(i, j), i \in \{0, \dots, R_p\}, j \in \{1, \dots, R_c\}$ that generates an exclusive combination of elements from plaintexts and ciphertexts.

For any two combinations of plaintexts and ciphertexts (m_a, c_a) and (m_b, c_b) , if $m_b > m_a$, then the condition $c_b > c_a$ must be met for the encryption function f to preserve order.

The OPE scheme of Boldyreva et al. [8] is the one on which most OPE research is based. This symmetric scheme comprises three algorithms (K_{gen}, Enc, Dec). Boldyreva et al. [8] showed that no efficient OPE system can be CPA (Chosen Plaintext Attack) safe, even when the ciphertext space is exponentially larger than the plaintext space. Several OPE techniques are inherently insecure as they leak significant extra information about their plaintexts from a collection of ciphertexts. Order-Revealing Encryption (ORE) [15] addresses this limitation by ensuring that ciphertexts reveal no more information than the ordering of their plaintexts. Specifically, in ORE the ciphertexts are non-numeric, and a comparison function is introduced to compare the ciphers. ORE encompasses three algorithms ($K_{gen}, Enc, Comp$):

- (1) A key generation algorithm K_{gen} that returns a secret key $\mathbb{K}$ based on the security parameter λ .
- (2) An encryption algorithm Enc that takes the secret key $\mathbb{K}$ and a plaintext m to return a ciphertext c .
- (3) A comparison algorithm $Comp$ that takes two ciphers (c_i, c_j) and returns $\mathbb{b} \in (0, 1)$ to denote whether c_i or c_j is bigger.

The encryption algorithm incorporates a parameter λ that governs the size of the key and ultimately the size of ciphertexts. For a message m of 8 bits, varying the parameter ' λ ' results in different sizes of ciphertexts—potentially 16-, 24-, 32-, 64-bits, and so forth. In the context of random tree inference, representing the ciphertexts as non-numeric and implementing the $Comp$ algorithm on an FPGA introduces significant overhead. Consequently, we convert non-numeric ciphers into numeric ones and employ arithmetic comparison operators ($>, \leq$) to determine the order of the ciphertexts. In Section 3.2 of [15], the authors recommend such conversion, particularly for applications where a numeric ciphertext space is more convenient and order relations can be computed without a "custom" comparison function $Comp$. Furthermore, such a conversion does not degrade the security of the encryption as outlined in [15] (Page 4, Section 1.1, second paragraph). For a full understanding of the ($K_{gen}, Enc, Comp$) algorithms, readers are referred to [15]. In [10], the OPE of [8] is applied to decision trees.

2.2 Encrypted Decision Nodes

Consider the training dataset $\mathbb{D} = [\mathbb{D}_x, \mathbb{D}_y]$, where $\mathbb{D}_x$ denotes the training samples and $\mathbb{D}_y$ denotes the class labels. In classification, the label of a leaf node corresponds to one of the labels in $\mathbb{D}_y$. Let L be the number of features, and denote the i -th feature as $\mathbb{D}_{x_i}$, $1 \leq i \leq L$. The number of unique values in $\mathbb{D}_y$ represents the number of class labels, denoted as ℓ . Assuming the trained RF model consists of $\mathbb{T}$ decision trees, consider one of the trees with N internal nodes: $I_1, I_2, \dots, I_N$, and $\mathbf{n}$ terminal or leaf nodes: $T_1, T_2, \dots, T_n$. The decision rule at I_k is encapsulated in the threshold

value Λ_k . The data feature $\mathbb{D}_{x_k}$ at node I_k and the threshold Λ_k define the inequality used in the decision rule of node I_k , expressed as

$$\mathbb{D}_{x_k} < \Lambda_k \text{ or } \mathbb{D}_{x_k} \geq \Lambda_k \quad (1)$$

The architecture of an RF model can be represented as $\mathbb{M}(\mathbb{T})$, where $\mathcal{M}_j(\mathbf{N}_j, \mathbf{n}_j) \in \mathbb{M}$ for $1 \leq j \leq \mathbb{T}$. Here, the index j denotes each decision tree out of $\mathbb{T}$ trees, and the notation $(\mathbf{N}_j, \mathbf{n}_j)$ represents the number of internal nodes and leaves of the j^{th} decision tree. RF inference involves traversing each decision tree from the root node to one of the leaf nodes. The internal nodes visited during this traversal are those defined by a chain of inequalities similar to (1). The inferred label is that of the leaf node at the end of the tree traversal, representing one of the class labels $b_j \in [0, \ell - 1]$ of the dataset for the j^{th} decision tree. The final outcome is determined by majority voting on the predictions of each decision tree:¹ $b = \text{majority voting}(b_1, b_2, \dots, b_j, b_{j+1}, \dots, b_{\mathbb{T}})$

During encryption, the feature and threshold associated with a specific decision rule at a given tree node are used. Encrypting the RF involves encrypting the computations at the internal nodes of each decision tree. Let $E_{\mathbb{K}}$ represent the OPE operator as described in [9]. The encrypted equivalents of the RF inequalities are then expressed as:

$$E_{\mathbb{K}}(\mathbb{D}_{x_k}) < E_{\mathbb{K}}(\Lambda_k) \text{ or } E_{\mathbb{K}}(\mathbb{D}_{x_k}) \geq E_{\mathbb{K}}(\Lambda_k) \quad (2)$$

Note that in the above inequalities, the order ($<, \geq$) of $\mathbb{D}_{x_k}$ with respect to Λ_k is preserved as in Eq. (1), i.e., $\mathbb{D}_{x_k} - \Lambda_k = \mathbf{D}_{\mathbb{K}}(E_{\mathbb{K}}(\mathbb{D}_{x_k}) - E_{\mathbb{K}}(\Lambda_k))$, where $\mathbf{D}_{\mathbb{K}}$ is the decryption operator using the $\mathbb{K}$ secret key. The secure representation of the RF must produce the correct inference result in encrypted form. Consequently, the labels of the leaf nodes T_k are also encrypted as $E_{\mathbb{K}}(T_k)$. The inferred class or label is subsequently determined using the private key $\mathbb{K}$ as follows:

$$T_k = \mathbf{D}_{\mathbb{K}}(E_{\mathbb{K}}(T_k)) \quad (3)$$

The majority voting rule is given by

$$b = \mathbf{D}_{\mathbb{K}}[\text{majority voting}\{E_{\mathbb{K}}(b_1), E_{\mathbb{K}}(b_2), \dots, E_{\mathbb{K}}(b_{\mathbb{T}})\}] \quad (4)$$

Training RF is carried out with plaintexts $\mathbb{D} = [\mathbb{D}_x, \mathbb{D}_y]$. After the RF is trained, OPE is applied as given in Eq. (2). The encryption of one of the trees $E_{\mathbb{K}}(\mathcal{M}_1(\mathbf{N}_1, \mathbf{n}_1))$ in RF is shown in Figure 1. Similar encryption is applied for all other trees $E_{\mathbb{K}}(\mathcal{M}_j(\mathbf{N}_j, \mathbf{n}_j))$, $2 \leq j \leq \mathbb{T}$.

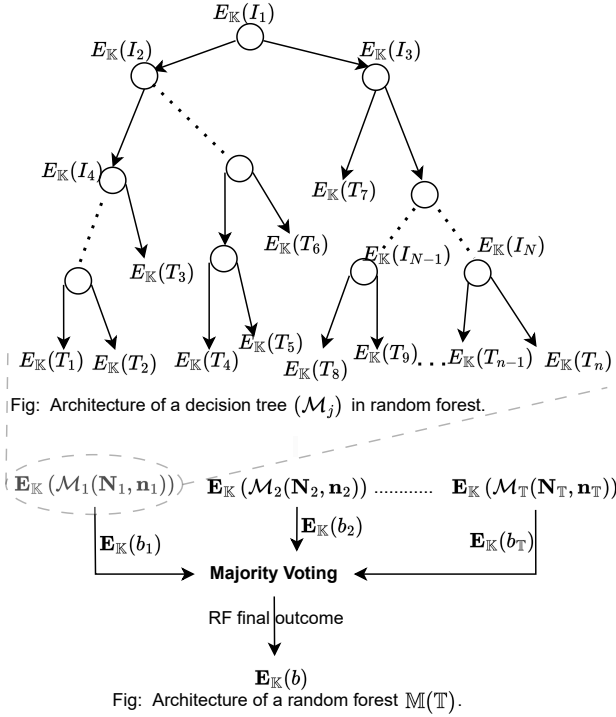
3 FPGA DESIGN OF RANDOM FOREST

In this investigation, Verilog is used as the hardware description language. Our FPGA implementation is categorized into two types of digital logic circuits: combinational and sequential. The combinational design of [12] and the sequential design of [10] are restricted to decision trees. In this paper, they are extended to RFs.

3.1 Edge Cloud Simulation

In the experimental testbed shown in Figure 2, an open source Python library *sklearn* [16] is used to train the RF model. Training and validation have been carried out on a compute intensive Amazon *m5d.2xlarge* EC2 virtual machine (VM). For OPE (modified ORE

¹The notation λ refers to the security parameter of the ORE cryptography while Λ refers to the threshold values of the RF model. Also, the notation $\mathbb{b}$ refers to outcome of $Comp$ operator of ORE while b refers to the inference outcome of the RF model.



as given in Section 2.1), we have used the open source code available at [17]. Once the training is completed, the decision rules from the root node to each leaf node are retrieved. These rules are then transformed into Verilog source code. For such a transformation, we use the tool described in [12]. The Quartus Prime Design Suite is used for the RTL synthesis of the RF’s Verilog source code and the generation of the SRAM-object file (sof). For the demonstration, we have used the well-known MNIST dataset [18], a collection of grayscale images of handwritten digits from 0 to 9, ($\ell = 10$). The features of the dataset are the pixels of the grayscale image with size $L = 28 \times 28 = 784$. For the FPGA implementation, we have selected the DE-10 Standard *SCSXFC6D6F31CN* FPGA board [19]. The FPGA acts as an edge device connected to the host machine through the UART port. After Quartus logic synthesis, the SRAM object file (sof) is sent over the USB port and loaded into the FPGA. Two inference experiments, *Expt1* and *Expt2* are carried out. In *Expt1*, the model is implemented on FPGA without any encryption, while in *Expt2*, the nodes and leaves of the RF model are encrypted as described in Section 2.2.

The diagram illustrates the proposed secure inference framework, showing the workflow from model training to secure inference.

Training Phase:

- A **Python Sklearn** server in the **cloud** is connected to a **Host machine** via **SSH Protocol** to connect and train the model.
- The **Host machine** runs **Quartus** and **Verilog**, which are connected to a **DE-10 Standard FPGA** via **USB 2.0**.
- The **Host machine** sends a **sof file** to the **DE-10 Standard FPGA** via **USB-A**.
- The **Host machine** and **DE-10 Standard FPGA** are connected via **UART Connection**.

Inference Phase:

- The **Host machine** sends **inference data** to the **DE-10 Standard FPGA** via **USB 2.0**.
- The **DE-10 Standard FPGA** returns **inferred class or label** to the **Host machine** via **UART Connection**.

Experimental Setup:

- Expt. 1:** Bob sends **Plain Random Forest model $M(\mathbb{T})$** to Alice via $\mathbb{D}_x$. Alice returns b to Bob.
- Expt. 2:** Bob sends **Encrypted Random Forest model $E_K[M(\mathbb{T})]$** to Alice via $-E_K(\mathbb{D}_x)$. Alice returns $-E_K(b)$ to Bob.

Figure 2: Experimental testbed of the edge-cloud environment.

We carry out all communication using the serial UART protocol. The connection diagram is shown in Figure 2. The MNIST pixels $E_{\mathbb{K}}(\mathbb{D}_{x_i}), 1 \leq i \leq L$ are transmitted from host machine to the FPGA while the RF’s inference result $E_{\mathbb{K}}(b)$ is sent by the FPGA to the host machine through *FT232RL*. The MNIST data samples have pixel values ≤ 255 . This causes the Λ thresholds of the decision node to be ≤ 255 too. With that specification, an 8-bit ($255 = 2^8 - 1$) representation of the decision tree model and inference data is adequate. However, the ciphers of nodes, leaves, and inference data may be more than 255. The cipher size ($|c|$) is determined by the OPE parameters λ given in [17].

$$N_w = \left\lfloor \frac{\lfloor \log_2(c) \rfloor}{8} \right\rfloor + 1 \quad (5)$$

3.3 Circuit Design of Random Forest

In [12], the combinational logic design of an encrypted decision tree is explored. The control flow structure of a tree in any programming language typically involves a sequence of nested "if-then-else" statements concluding with a clause that represents the class label outcome of the leaf node. Such conditional statements are equivalent to the inequality relationships presented in (1) and (2). These inequalities, corresponding to the nodes between the root and any

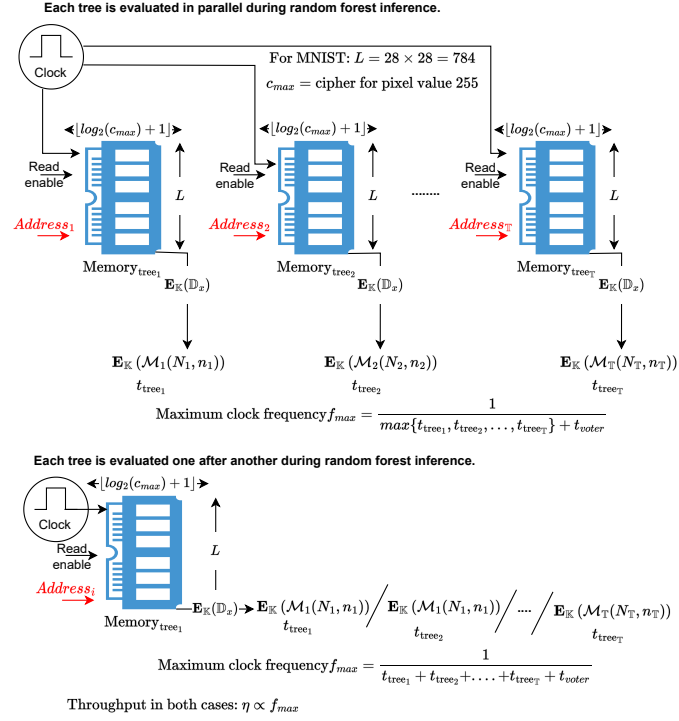


Figure 3: FPGA memory map showing parallel vs successive processing of trees during the random forest inferencing.

given leaf, result in a Boolean function that mirrors the structure of the tree as shown in Figure 1. For example,

$$\begin{aligned} E_K(T_1) &= E_K(I_1) \& E_K(I_2) \& E_K(I_4) \& \dots \\ E_K(T_3) &= E_K(I_1) \& E_K(I_2) \& \overline{E_K(I_4)} \end{aligned}$$

where “&” represents the logical AND operation.

In [10], an alternative sequential logic design is suggested where the transition graph of its finite-state machine (FSM) mirrors that of the decision tree (DT), with each tree node representing one FSM state. During each clock cycle, one of the FSM states is assessed by examining a single inequality relation, as described in (2). In the hardware, this comparison is realized through a comparator and a multiplexer. The comparison attributes, namely the dataset feature $E_K(\mathbb{D}_{x_i})$ for each decision tree node, are stored in a RAM. Inference data samples are retrieved from the RAM and input to the root node I_1 of each decision tree. Subsequent states are determined in the following clock cycle after computing the inequality operation of I_1 . This iterative process continues until the evaluation reaches a leaf state, i.e., an encrypted terminal node $E_K(T_i)$. Upon reaching a leaf state, the class/label of $E_K(T_i)$ is extracted and returned as the inference result of that particular DT. The same processing applies to other DT's of the RF model.

This paper employs similar combinational [12] and sequential [10] logic circuits to implement each tree. The width (or number of bits) of the RAM is set to match the size of the maximum value of the ciphertext c_{max} . In OPE, this ciphertext corresponds to the maximum value of the MNIST pixel, which is 255.

3.4 Parallel vs. Sequential Tree Evaluation

The length (or the number of addresses) of the RAM equals the number of features in the dataset, namely, $L = 28 \times 28 = 784$ for the MNIST dataset. The visual representation is shown in Figure 3. Two designs of the memory map are presented. In the first architecture, T copies of RAM are utilized, with one RAM allocated to inferencing one tree in the RF. All the copies of RAM are triggered by the same clock frequency. But the memory address of each RAM is different. For example, assume $T = 3$, and consider the following computation at the root node:

$$\begin{aligned} [E_K(I_1)]_{tree1} &: E_K(\mathbb{D}_{x_i}) < E_K(\Lambda_i) \\ [E_K(I_1)]_{tree2} &: E_K(\mathbb{D}_{x_j}) < E_K(\Lambda_j) \\ [E_K(I_1)]_{tree3} &: E_K(\mathbb{D}_{x_k}) < E_K(\Lambda_k) \end{aligned}$$

In this context, the indices i, j, k denote the i^{th} , j^{th} , and k^{th} pixel values within the range of 0 to 255. These indices correspond to RAM addresses $\{Address_1, Address_2, Address_3\}$ highlighted in red in Figure 3. Similar mapping extends to other nodes of each tree. Given the constraint of only one memory read per clock cycle in the specified FPGA device, it becomes imperative to allocate a dedicated memory for each tree to enable concurrent processing of multiple trees. Consequently, the RAM has been scaled for each tree to accommodate such requirement. In this scenario, the FPGA resources in terms of flip-flops, logic blocks, and DSP slices increase with the number of trees. As each tree is computed in parallel, the utilization of AND gates in the combinational design and Comparators/Multiplexers in the sequential design scales up linearly with respect to T .²

The second architecture employs only one RAM, and the trees are processed sequentially. After one tree produces the inference b_j , it is stored in a register, and the processing of the next tree commences, as illustrated in Figure 3. Once all the trees have completed their inferences, the final outcome is determined using the majority voting mechanism. Clearly, the second architecture would require much more time to complete the inference. The maximum clock frequency at which the overall design can operate is significantly lower in the second architecture than in the first. Additionally, this clock frequency degrades when scaling up the number of trees. Ultimately, the clock frequency determines the inference throughput.

Clearly, there is a trade-off between the throughput and FPGA resource utilization. In this work, an edge-cloud environment is considered, in which acceleration is prioritized. Consequently, we choose the first design, which involves concurrently computing trees in a RF. In either design, inference data samples are read from RAM and fed into the tree root node $E_K(I_1)$. The cipher thresholds $E_K(\Lambda)$'s are hardcoded in the Verilog script for both the combinational and sequential design. The subsequent nodes, $E_K(I_2)$ or $E_K(I_3)$, are evaluated based on the query outcome of the previous node. Once the leaf is reached, the class/label of $E_K(T_i)$ is returned as the inference result of that particular tree. The majority voting is applied over all the tree's outcomes through the combinational circuit; an example for $T = 3$ is shown in Fig 4. The circuit diagram is constructed

²A pipelined design may be employed instead of scaling the RAM. Indeed, memory access could be pipelined with the comparison operations ($\leq$, $>$). The High-Level Synthesis (HLS) tool, integrated with OPE, can be employed to generate the optimized design. We expect to report on the pipelined design in a forthcoming publication.

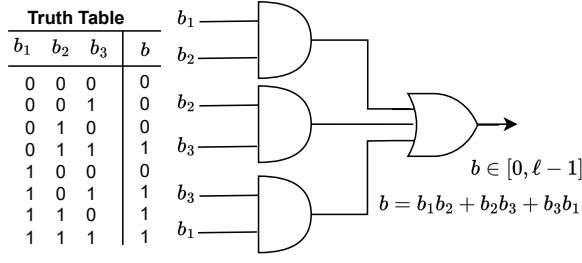


Figure 4: Majority voting over each $M_j(N_j, n_j)$'s outcome b_j 's to produce b , the final inference outcome of random forest model $\mathbb{M}(\mathbb{T})$.

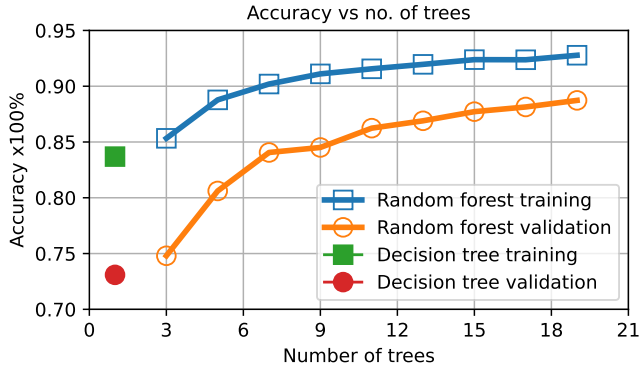


Figure 5: Accuracy of a decision tree (DT) and random forest (RF) with various values of $\mathbb{T}$. For each tree, the limit is set such that $N_j \leq 600, n_j \leq 300, 1 \leq j \leq \mathbb{T}$, and maximum 20 nodes between root and a leaf across all trees. The accuracies of plain and encrypted RF/DT are the same.

based on the truth table, representing all possible combinations for various outputs of each tree during RF inference. Consider the case where $b_1 = b_2 = \ell$, then the majority voting $b = \ell(1 + b_3 + b_3) = \ell$, thus the Boolean expression produces the correct inferential outcome. Similarly, one could test other scenarios of the truth table. For larger number of trees, one may consider adopting the hardware design using multiplexers.

4 EXPERIMENTAL EVALUATION

4.1 FPGA Resource Analysis

We evaluated the inference accuracy across various decision tree architectures, as depicted in Figure 5. The chosen architecture features $N_j = 600$ nodes and $n_j = 300$ leaves, with a total of $\mathbb{T} = 3$ trees, where $1 \leq j \leq \mathbb{T}$.³ This parameter selection resulted in an accuracy of 86.12% on the validation dataset. Two inference experiments, denoted as *Expt1* and *Expt2*, were conducted as explained in Section 3.1. A comparison of FPGA resources is presented in Tables 1 and 2. The percentage utilization in Tables 1 and 2 represents

³For illustrative purposes, we have used the MNIST dataset and implemented an RF with three DTs. Our methodology is adaptable to any dataset, with the number of DTs determined through hyperparameter tuning, as shown in Fig. 5, provided that the FPGA resources can accommodate the specified number of trees.

the fraction of total ALMs (adaptive logic modules), LABs (logic array blocks), and FFs (dedicated primary registers) of the DE-10 standard FPGA used in the RF implementation. The FPGA clock frequency is set to 50 MHz. As anticipated, the resources consumed by the encrypted RF model exceed those of the plain one. Of particular interest is the use of LABs and FFs in the encrypted case due to additional storage requirements. Such use may contribute to increased power consumption during additional READ operations corresponding to large ciphertexts. The functionality and performance of the design are determined by the slack setup/hold time, both of which are positive in our case. Three tests were conducted for the encrypted scenario, employing different ciphertext sizes by varying the OPE security parameter λ . This resulted in ciphertext lengths of {18, 32, and 64}-bits, corresponding to $N_w = \{3, 4, \text{ and } 8\}$, respectively. During Quartus synthesis of the behavioral Verilog code corresponding to the RF model, the resource consumption of ALMs and LABs exceeded 100%, indicating that the available FPGA resources for the specified FPGA were insufficient to store and operate the RF design. Consequently, the synthesis failed to complete the run, and timing analysis results were not returned by Quartus. For such designs, the outcomes are marked as 'x' in Tables 1 and 2. An alternative approach to resolve the 'x' cases is to sacrifice throughput and evaluate the trees serially (Figure 3).

Surprisingly, the sequential design outperformed the combinational design as it successfully completed the synthesis of the RF design with 32-bit ciphertexts. Because the sequential design is

Table 1: FPGA resource comparison for random forest inference with $\mathbb{T} = 3$ in combinational design (design circuit available in [12]). ALMs: Adaptive logic modules, LABs: Logic array blocks, ALUTs: Combinational adaptive look-up tables, FFs: Dedicated primary registers, F_{max} : Maximum attainable clock frequency, Setup: Slack time for clock setup, Hold: Slack time for clock hold.

Attributes	Unencrypted (Fig. 2 Expt1)	OPE as per Section 2.1 (Fig. 2 Expt2)		
		$N_w = 8$ ($ c = 64$)	$N_w = 4$ ($ c = 32$)	$N_w = 3$ ($ c = 24$)
ALMs (%)	9	136	122	75
LABs (%)	13	136	123	97
ALUTs	5795	63509	54033	28670
FFs (%)	4	90	87	66
F_{max} (MHz)	68.92	x	x	62.11
Setup (ns)	5.491	x	x	3.899
Hold (ns)	0.372	x	x	0.344

Table 2: FPGA resource comparison for random forest inference with $\mathbb{T} = 3$ in sequential design (design circuit available in [10]).

Attributes	Unencrypted (Fig. 2 Expt1)	OPE as per Section 2.1 (Fig. 2 Expt2)		
		$N_w = 8$ ($ c = 64$)	$N_w = 4$ ($ c = 32$)	$N_w = 3$ ($ c = 24$)
ALMs (%)	15	140	49	41
LABs (%)	23	142	68	55
ALUTs	9960	69129	31221	27547
FFs (%)	4	89	30	25
F_{max} (MHz)	71.73	x	61.7	71.2
Setup (ns)	5.578	x	3.793	4.987
Hold (ns)	0.371	x	0.365	0.368

triggered at the clock edge, the computation in just one state of the FSM is completed in one clock cycle. The computation performed in each clock cycle can be precisely regulated using the clock. Such clock control is not accessible in the combinational design, and the entire tree computation is triggered by any change in the module inputs. Consequently, significant FPGA resource usage occurs in the combinational design. Additionally, the value of F_{max} indicates that the sequential design has the ability to operate at a higher clock frequency. Therefore, the sequential design is favored for confidential inference.

4.2 Discussion

In OPE, the size of ciphertexts plays a pivotal role in fortifying cryptographic strength. Larger ciphertexts size $|c|$ adds complexity, rendering traditional cryptanalysis techniques more challenging for potential adversaries. This increased complexity offers protection against frequency analysis, where the distribution of plaintext values could be exploited. Moreover, longer ciphertexts bolster resistance against dictionary attacks, as the computational burden of precomputing exhaustive tables becomes impractical. The expanded search space provided by larger ciphertexts enhances security by making it more harder for attackers to guess or infer plaintext values based on partial information. It is therefore advisable to utilize a considerably larger value of $|c|$ that can fit within the FPGA.

The trend illustrated in Fig. 5 reveals that augmenting the number of trees, denoted as $\mathbb{T}$, leads to increased accuracy. However, incorporating numerous trees may prove inadequate for FPGA resources. According to Table 2, the maximum 32-bit ciphertext size can be accommodated, but this comes at the cost of utilizing approximately half of the ALMs and LABs. Introducing additional trees, such as $\mathbb{T} = 5, 7, 9, 11, \dots$, becomes impractical. The demonstration illustrating this is presented in Table 3. Despite compromising the security strength, i.e., setting the cipher size to $|c| = 16$ bits, and using only 9 trees, more than half of the resources are already consumed. The model developer is expected to take into account the desired RF's inference accuracy while adhering to FPGA resource constraints along with security strength.

5 CONCLUSIONS

This paper presented the first use of OPE for the confidential inference in an RF model. Parallel and sequential tree evaluations

Table 3: FPGA resource comparison for inferencing over different random forest architecture with $|c| = 16$ -bits (or $N_w = 2$) in sequential design (design circuit available in [10]).

Attributes	OPE as per Section 2.1 (Fig. 2 Expt2)			
	$\mathbb{T} = 3$ Acc $\approx 75\%$	$\mathbb{T} = 5$ Acc $\approx 80\%$	$\mathbb{T} = 7$ Acc $\approx 84\%$	$\mathbb{T} = 9$ Acc $\approx 85\%$
ALMs (%)	25	47	58	67
LABs (%)	33	52	72	79
ALUTs	14327	23125	29134	35123
FFs (%)	17	29	42	53
F_{max} (MHz)	71.4	70.6	67.7	66.3
Setup (ns)	5.182	5.053	4.019	4.002
Hold (ns)	0.37	0.362	0.358	0.342

are used to achieve a resource-efficient FPGA implementation of the encrypted RF. Additionally, a serial communication protocol is specifically designed to streamline hardware-accelerated inference in edge-cloud environments. In an upcoming publication, we will develop various attacks on the OPE of RFs and assess its resilience using standard cybersecurity benchmarks.

ACKNOWLEDGMENT

This research has been conducted at Khalifa University with financial support provided by the Technology Innovation Institute (TII), Abu Dhabi, UAE, under contract TII/CRP/2036/2020. The authors are grateful to Elena Kirshanova, Nitin Satpute, and Jinson Thomas, all from TII, for enlightening discussions and helpful comments. Rupesh Raj Karn is now with New York University, Abu Dhabi, UAE.

REFERENCES

- [1] Qianying Liao, Bruno Cabral, João Paulo Fernandes, and Nuno Lourenço. Herb: Privacy-preserving random forest with partially homomorphic encryption. In *2022 International Joint Conference on Neural Networks (IJCNN)*, pages 1–10. IEEE, 2022.
- [2] Yuncheng Wu, Shaofeng Cai, Xiaokui Xiao, Gang Chen, and Beng Chin Ooi. Privacy preserving vertical federated learning for tree-based models. *arXiv preprint arXiv:2008.06170*, 2020.
- [3] Zhuoran Ma, Jianfeng Ma, Yinbin Miao, and Ximeng Liu. Privacy-preserving and high-accurate outsourced disease predictor on random forest. *Information Sciences*, 496:225–241, 2019.
- [4] Nicholas Carlini et. al. The secret sharer: Evaluating and testing unintended memorization in neural networks. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 267–284, 2019.
- [5] Zhanglong Ji, Zachary C Lipton, and Charles Elkan. Differential privacy and machine learning: a survey and review. *arXiv preprint arXiv:1412.7584*, 2014.
- [6] Asma Aloufi et. al. Blindfolded evaluation of random forests with multi-key homomorphic encryption. *IEEE Transactions on Dependable and Secure Computing*, 18(4):1821–1835, 2019.
- [7] David J Wu, Tony Feng, Michael Naehrig, and Kristin Lauter. Privately evaluating decision trees and random forests. *Cryptology ePrint Archive*, 2015.
- [8] Alexandra Boldyreva et. al. Order-preserving symmetric encryption. In *Advances in Cryptology-EUROCRYPT 2009: 28th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cologne, Germany, April 26-30, 2009. Proceedings 28*, pages 224–241. Springer, 2009.
- [9] Kevin Lewi and David J Wu. Order-revealing encryption: New constructions, applications, and lower bounds. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 1167–1178, 2016.
- [10] Rupesh Raj Karn, Kashif Nawaz, and Ibrahim M Elfadel. Securing decision tree inference using order-preserving cryptography. In *5th IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS 2023)*, pages 1–5. IEEE, 2023.
- [11] Craig Gentry. *A fully homomorphic encryption scheme*. Stanford university, 2009.
- [12] Rupesh Raj Karn and Ibrahim M Elfadel. Confidential inference in decision trees: FPGA design and implementation. In *30th IFIP/IEEE International Conference on Very Large Scale Integration (VLSI-SoC 2022)*, pages 1–6. IEEE, 2022.
- [13] Sioni Summers, Giuseppe Di Guglielmo, Javier Duarte, Philip Harris, Duc Hoang, Sergio Jindariani, Edward Kreinar, Vladimir Loncar, Jennifer Ngadiuba, Maurizio Pierini, et al. Fast inference of boosted decision trees in fpgas for particle physics. *Journal of Instrumentation*, 15(05):P05026, 2020.
- [14] Rafal Kulaga and Marek Gorgon. FPGA implementation of decision trees and tree ensembles for character recognition in vivo. *Image Processing & Communications*, 19(2-3):71, 2014.
- [15] Nathan Chenette et. al. Practical order-revealing encryption with limited leakage. In *Fast Software Encryption: 23rd International Conference, FSE 2016, Bochum, Germany, March 20-23, 2016*, pages 474–493. Springer, 2016.
- [16] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *the Journal of machine Learning research*, 12:2825–2830, 2011.
- [17] Kevin Lewi David Wu. Fastore.
- [18] Yann LeCun et. al. Gradient-based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [19] DE-10 standard FPGA. <http://de10-standard.terasic.com/>.